

Información General

Curso : Aprendizaje profundo
Semestre : 2026 - 1
Profesor : Gibrán Fuentes Pineda
Entrega : Noviembre 9 de 2025
Alumno : Pablo Uriel Benítez Ramírez, 418003561

Tarea: Redes convolucionales y

recurrentes

1. Operación de convolución

Extiende la operación de convolución para imágenes en escala de grises (un solo canal) vista en clase (https://github.com/gibranfp/CursoAprendizajeProfundo/blob/2026-1/notebooks/2a_convolucion.ipynb) a imágenes a color (múltiples canales).

Procedimiento 1.1. El cambio de escala de grises a color implica la modificación de un solo canal a multiples canales. Cada filtro tiene en la mira a todos los canales de entrada y los combina, lo que provoca que el resultado para cada canal de salida sea la suma de las convoluciones por canal más un sesgo. La operación de convolución entre una imagen I y un filtro W , la cual está definida por

$$A_{i,j} = (\mathbf{I} * \mathbf{W})_{i,j} = \sum_m \sum_n I_{m,n} W_{i-m,j-n}$$

La convolución es commutativa, por lo tanto

$$A_{i,j} = (\mathbf{W} * \mathbf{I})_{i,j} = \sum_m \sum_n I_{i-m,j-n} W_{m,n}$$

Para este caso con múltiples canales, para una imagen $I \in \mathbb{R}^{H \times W \times C}$ y un filtro $W \in \mathbb{R}^{k \times k \times C}$ la operación es

$$A_{i,j} = (\mathbf{W} * \mathbf{I})_{i,j} = \sum_{c=1}^C \sum_{m=1}^k \sum_{n=1}^k I_{i+m,j+n,c} W_{m,n,c}$$

El resultado de estas operaciones es el mapa de activaciones $A(i,j)$.

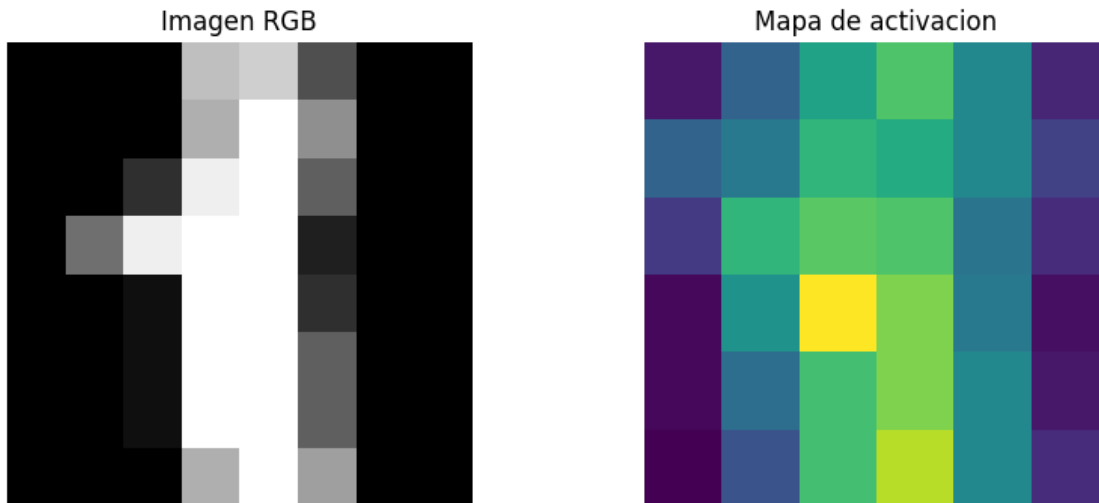


Figura 1: Mapa de activacion

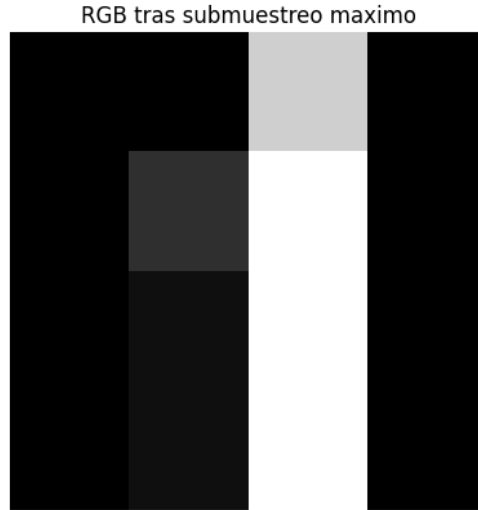


Figura 2: RGB tras submuestreo maximo

Código 1.2.  *Nombre de archivo:* 1_Operación_de_convolución.ipynb

2. Diferenciación automática

Considera una red sin sesgos compuesta por una sola capa convolucional estándar con un filtro y una capa de salida con una neurona. Realiza un paso de actualización del descenso por gradiente estocástico calculando la propagación hacia adelante y hacia atrás a partir de lo siguiente:

- Imagen de 4×4

$$\mathbf{I} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- Filtro de 2×2 con desplazamiento de 2 y sin relleno

$$\mathbf{K} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$$

- Pesos de la neurona de salida

$$\mathbf{w} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

- Salida $y = 1$
- Función de activación ReLU a la salida de la capa convolucional y sigmoide en la neurona de salida
- Función de pérdida de entropía cruzada binaria
- Tasa de aprendizaje $\alpha = 0.001$

Propagación hacia adelante

Se aplica el filtro K sobre la imagen de entrada I usando un desplazamiento de 2 y sin padding, se genera un mapa de activaciones con 4 valores.

Se aplica la función de activación ReLU a las salidas de la convolución, se reemplazan los valores negativos por cero:

$$\text{ReLU}(x) = \max(0, x)$$

Capa densa, se conectan las 4 activaciones de ReLU a una neurona de salida mediante los pesos w .

$$z = w^T \cdot \text{ReLU}$$

Función sigmoide, la salida de la capa densa pasa por la función sigmoide para producir la predicción $\hat{y}$:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Función de pérdida se utiliza entropía cruzada binaria:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Salida:

Propagación hacia adelante

Salida de convolucion: $[[0 \ 1 \ 1 \ 0]]$

Salida de ReLU: $[[0 \ 1 \ 1 \ 0]]$

Salida de la red (sigmoid): $[[0.88079708]]$

Pérdida (cross-entropy): $[[0.12692801]]$

Retropropagación

Gradiente de la pérdida respecto a la salida lineal

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y$$

Gradiente respecto a los pesos de salida

$$\frac{\partial \mathcal{L}}{\partial w} = (\hat{y} - y) \cdot \text{ReLU}$$

Gradiente respecto a la salida de ReLU

$$\frac{\partial \mathcal{L}}{\partial \text{ReLU}} = (\hat{y} - y) \cdot w$$

Gradiente respecto a la salida de la convolución usando la derivada de ReLU:

$$\frac{\partial \mathcal{L}}{\partial \text{conv}} = \frac{\partial \mathcal{L}}{\partial \text{ReLU}} \cdot \mathbb{I}_{\text{conv} > 0}$$

Gradiente respecto al filtro K

$$\frac{\partial \mathcal{L}}{\partial K} = \sum_{i=1}^4 \delta_i \cdot \text{patch}_i$$

donde δ_i es el gradiente en cada posición de la convolución. Actualización de parámetros:

$$w \leftarrow w - \alpha \cdot \frac{\partial \mathcal{L}}{\partial w} \quad \text{y} \quad K \leftarrow K - \alpha \cdot \frac{\partial \mathcal{L}}{\partial K}$$

Salida:

```

Retropropagación
Gradiente respecto a w: [[-0.          -0.11920292 -0.11920292 -0.          ]]
Gradiente respecto a K: [[ 0.          -0.23840584]
                          [-0.23840584  0.          ]]

```

```

Pesos actualizados w:
[[0.          ]
 [1.0001192]
 [1.0001192]
 [0.          ]]
Kernel actualizado K:
[[1.00000000e+00 2.38405844e-04]
 [1.00023841e+00 0.00000000e+00]]

```

2.1. Desarrollo Analítico de la Diferenciación Automática

Sea una red compuesta por una imagen de entrada I de tamaño 4×4 , filtro K de tamaño 2×2 con desplazamiento 2, sin padding, la activación ReLU después de la capa convolucional y sigmoide al final; y una capa densa de una neurona con pesos $\mathbf{w} \in \mathbb{R}^4$, la función de pérdida es entropía cruzada binaria, con tasa de aprendizaje $\alpha = 0.001$

2.1.1. Propagación hacia adelante

convolución con desplazamiento 2

$$A = \text{ReLU}(I * K) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Aplanando la activación: $\mathbf{a} = [0, 1, 1, 0]$, la neurona de salida es

$$\mathbf{w} = [0, 1, 1, 0]$$

el logit

$$z = \mathbf{w} \cdot \mathbf{a} = 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 1 + 0 \cdot 0 = 2$$

sigmoide

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-2}} \approx 0.8808$$

por último la función de la pérdida

$$\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}) = -\log(0.8808) \approx 0.1269$$

2.1.2. Propagación hacia atrás

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\frac{1}{\hat{y}} \approx -1.135$$

$$\frac{d\hat{y}}{dz} = \hat{y}(1 - \hat{y}) \approx 0.8808 \cdot 0.1192 = 0.105$$

$$\frac{\partial \mathcal{L}}{\partial z} = -1.135 \cdot 0.105 \approx -0.119$$

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} \cdot a_i \Rightarrow \nabla_w = -0.119 \cdot [0, 1, 1, 0] = [0, -0.119, -0.119, 0]$$

actualización de pesos

$$w_{\text{nuevo}} = w - \alpha \cdot \nabla_w = [0, 1, 1, 0] - 0.001 \cdot [0, -0.119, -0.119, 0] = [0, 1.000119, 1.000119, 0]$$

quedando como

Activación de la convolución: $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ Predicción: $\hat{y} \approx 0.8808$ Pérdida: $\mathcal{L} \approx 0.1269$ Nuevos pesos: $[0, 1.000119, 1.000119, 0]$

Código 2.1.  Nombre de archivo: 2.Diferenciación_automática.ipynb

3. Clasificación de rostros por grupo etario

Entrena y evalúa modelos de clasificación de rostros por grupo etario usando el conjunto de datos FairFace1. Para ello, es necesario:

- Dividir aleatoriamente el conjunto de entrenamiento en subconjuntos de entrenamiento y validación.
- Programar tu propia clase para la carga de datos.
- Agregar acrecentamiento de datos.
- Entrenar un modelo similar al de la libreta https://github.com/gibranfp/CursoAprendizajeProfundo/blob/2025-1/notebooks/2b_convnet_resnet.ipynb, pero cambiando los bloques ResNet por bloques ConvNeXt.

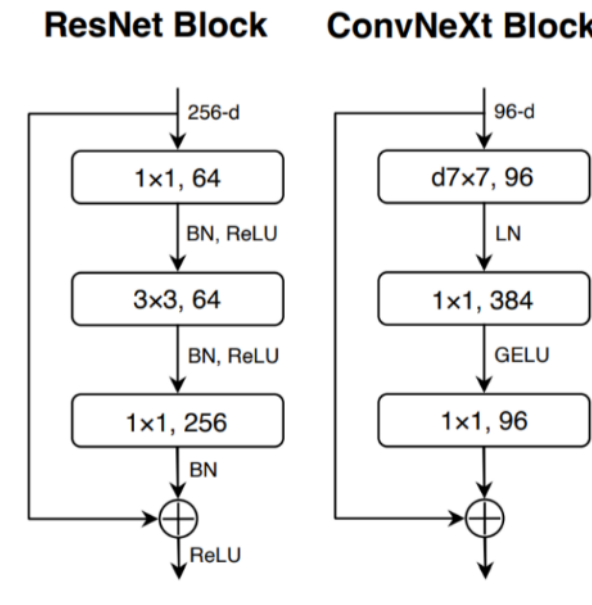


Imagen tomada de Liu et al. (2022). A ConvNet for the 2020s.

- Entrenar un modelo mediante aprendizaje por transferencia a partir de alguna red preentrenada actualizando solo los parámetros de la capa de salida y otro modelo actualizando los parámetros de toda la red.
- Graficar las pérdidas por época en los conjuntos de entrenamiento y validación.
- Evaluar los modelos usando métricas apropiadas para la tarea y sus características (por ej. si hay desbalance entre clases).
- Discutir los resultados.

4. Reconocimiento de comandos de voz

Se desarrollaron e implementaron tres modelos de redes neuronales para la tarea de reconocimiento de comandos de voz utilizando el conjunto de datos speech commands. Las entradas al modelo fueron representaciones espectrales del audio, concretamente MFCCs, extraídas de las grabaciones.

- Entrenar un modelo basado en redes recurrentes.
- Entrenar un modelo basado en capas convolucionales 1D.
- Entrenar un modelo basado en capas convolucionales 2D.
- Graficar las pérdidas por época en los conjuntos de entrenamiento y validación.

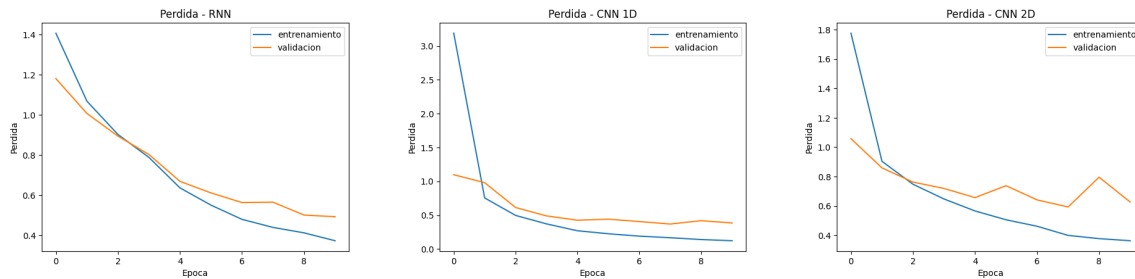


Figura 3: Visualización de las pérdidas por época en los conjuntos de entrenamiento y validación del modelo basado en redes recurrentes, capas convolucionales 1D, capas convolucionales 2D respectivamente.

- Evaluar los modelos usando métricas apropiadas para la tarea y sus características (por ej. si hay desbalance entre clases). Se evalúa por F1-score y muestra los errores por clase.
- Discutir los resultados. En general, todos los modelos lograron reconocer comandos de voz con buen nivel de precisión, pero las redes convolucionales mostraron mejores resultados que la red recurrente. La CNN 2D fue la que mejor funcionó, probablemente porque aprovecha bien la estructura de los MFCCs como si fueran imágenes. La CNN 1D también tuvo buen rendimiento y es más sencilla de entrenar. Las RNN son útiles, pero más costosas computacionalmente.

En todos los casos, se representan los comandos de voz mediante MFCCs.

Código 4.1.  *Nombre de archivo:* 4_Reconocimiento_de_comandos_de_voz.ipynb